

面向对象的设计原则

目的：支持可维护的复用

- 目标：开闭原则
- 指导：最小知识原则
- 基础：单一职责原则、可变性封装原则
- 实现：依赖倒置原则、合成复用原则、里氏代换原则、接口隔离原则

设计原则名称	设计原则简介	设计原则之间关系
单一职责原则 SRP Single Responsibility Principle	一个对象应该只包含 单一的职责 ，并且该职责被完整地封装在一个类中。	
开闭原则 OCP Open-Closed Principle	一个软件实体应该 对扩展开放，对修改关闭 。软件实体可以是一个软件模块、一个由多个类组成的局部结构或一个单独的类。	开闭原则是对单一职责原则的加强。
里氏代换原则 LSP Liskov Substitution Principle	所有引用基类（父类）的地方必须能 透明地 使用其子类的对象。 通俗表达：软件中如果能够使用基类对象，那么一定能够使用其子类对象。	里氏代换原则是开闭原则的具体实现。
依赖倒转原则 DIP Dependency Inversion Principle	高层模块 不应该依赖于低层模块 ，他们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。 另一种表述：要针对接口编程，而不要针对实现编程。	依赖倒转原则是开闭原则的具体实现。
接口隔离原则 ISP Interface Segregation Principle	客户不应该依赖那些它不需要的接口。	拆分时需要满足单一职责原则
合成复用原则 CRP Composite Reuse Principle	在系统中应该尽量多实用组合聚合关联关系，尽量少甚至不使用继承关系。	
迪米特法则 LoD Law of Demter	在一个软件实体对其他实体的引用越少越好，或者说如果两个类不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，而是通过引入一个第三者发生简介交互。	

中文名称	英文名称	内容	关系
最小知识原则	Least Knowledge Principle, LKP?	就是迪米特法则	
可变性封装原则	Principle of Encapsulation of Variation, EVP	找到一个系统的可变因素，将其封装起来	是开闭原则的一种具体实现

设计模式

定义（什么是设计模式？）

- 设计模式是对被用来在特定场景下解决一般设计问题的类和互相通信的对象的描述，包括**设计名称（Pattern Name）**、**问题（Problem）**、**解决方案（Solution）**、**效果（Consequences）**。
- （PPT）设计模式（Design Pattern）是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

分类

分类一

- 创建型模式 (Creational)：主要用于创建对象；例如：工厂方法模式、抽象工厂模式
- 结构型模式 (Structural)：主要用于处理类或对象的组合；例如：适配器模式、组合模式、外观模式、装饰模式
- 行为性模式 (Behavioral)：主要用于描述对类或对象怎样交互和怎样分配职责，例如：模板方法模式、命令模式、中介者模式、观察者模式

分类二 (copied from dxy)

- 类模式：处理类和子类之间的关系，这些关系通过继承建立，在编译时刻就被确定下来，是属于静态的；例如：工厂方法模式、（类）适配器模式、模板方法模式。
- 对象模式：处理对象间的关系，这些关系在运行时刻变化，更具动态性；例如：抽象工厂模式、（对象）适配器模式、命令模式、中介者模式、观察者模式。

绝大多数都属于对象模式，很少有类模式。

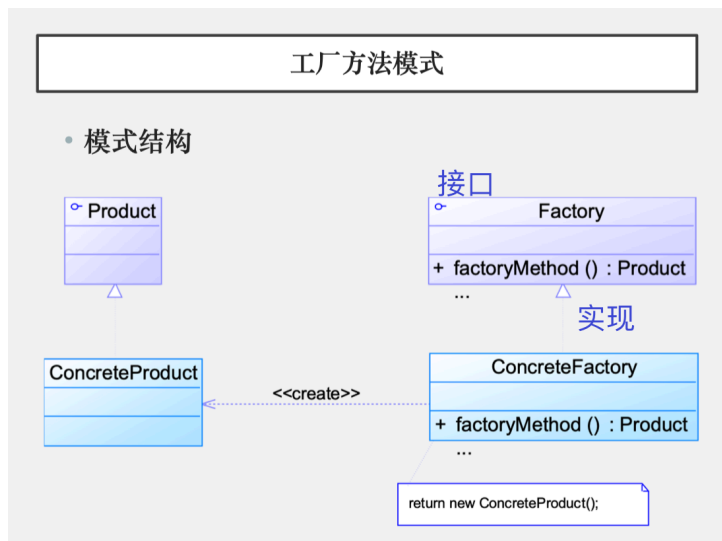
范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	模板方法模式
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 享元模式 代理模式	命令模式 迭代器模式 中介者模式 观察者模式 状态模式 策略模式

具体设计模式

工厂方法模式 (Factory Method Pattern)

又称为：

- 工厂模式
- 虚拟构造器 (Virtual Constructor) 模式
- 多态工厂 (Polymorphic Factory) 模式



属于类模式，创建型模式。

在工厂方法模式中，核心的工厂类不再负责所有产品的创建，而是将具体创建工作交给子类去做；可以允许系统在不修改工厂角色的情况下引进新产品。

优点

- 需要添加产品对象时，只需要添加一个具体产品对象和一个具体工厂对象，不需要对原有实现修改，很好符合**开闭原则**（克服了简单工厂模式的缺点）
- 工厂方法模式退化后可以演变成简单工厂模式。

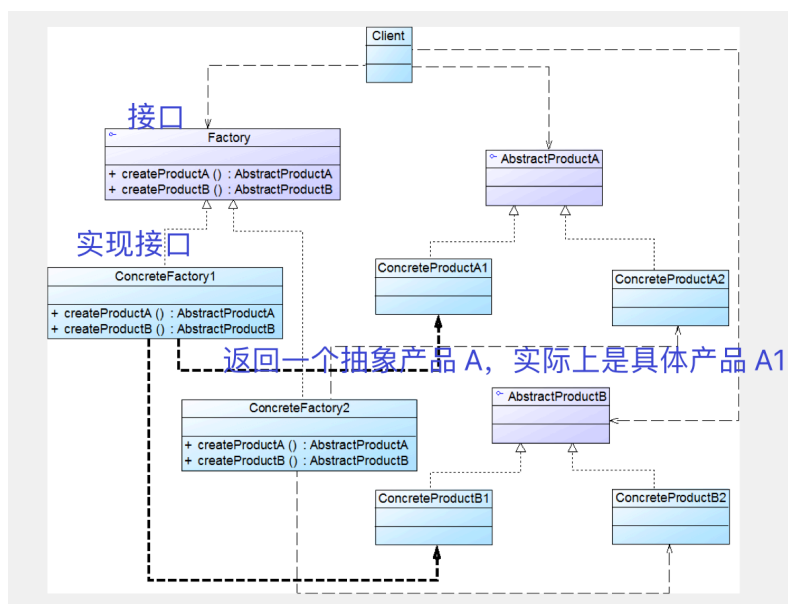
缺点

- 类太多，系统复杂度增加，为系统增加一些额外的开销。
- 增加了系统的抽象性和理解难度、实现难度等。

抽象工厂模式 (Abstract Factory Pattern)

又称为：Kit 模式

属于对象创建型模式。

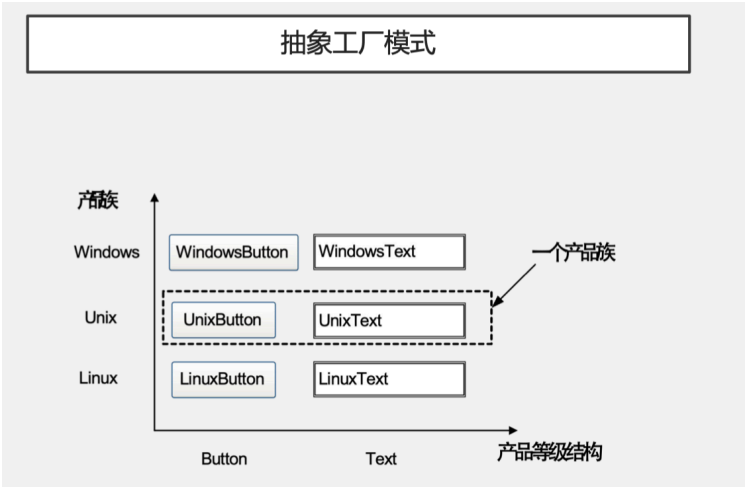


产品等级结构（继承关系）

产品等级结构即产品的继承结构。如一个抽象类是电视机，其子类有海尔电视机、海信电视机、TCL 电视机，则抽象电视机与具体品牌的电视机之间构成了一个产品等级结构，抽象电视机是父类，而具体品牌的电视机是其子类。

产品族

是指由同一个工厂生产的，位于不同产品等级结构的一组产品。如海尔电器工厂生产的海尔电视机、海尔电冰箱，海尔电视机位于电视机产品等级结构中，海尔电冰箱位于电冰箱产品等级结构中。



优点

- 抽象工厂模式隔离了具体类的生成。只需改变具体工厂的实例，就可以在某种程度上改变整个软件系统的行为（从一个品牌的所有产品换到另一个品牌的所有产品）。
- 能够保证客户端始终只使用他同一个产品族中的对象。
- 增加新的具体工厂和产品族很方便，无须修改已有系统，符合“开闭原则”。

缺点

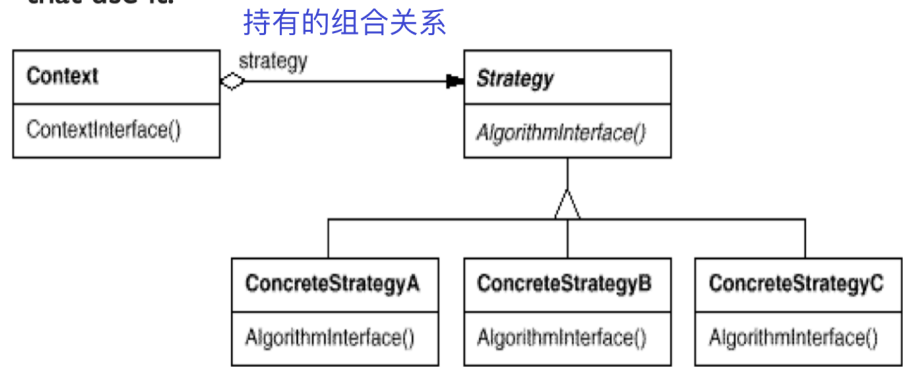
- 在添加新的产品对象时，难以扩展抽象工厂来生产新种类的产品（**开闭原则的倾斜性**：增加新的工厂和产品族容易，增加新的产品等级结构麻烦）

策略模式（Strategy Pattern）

又称：Policy

是对象行为型模式。

- The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

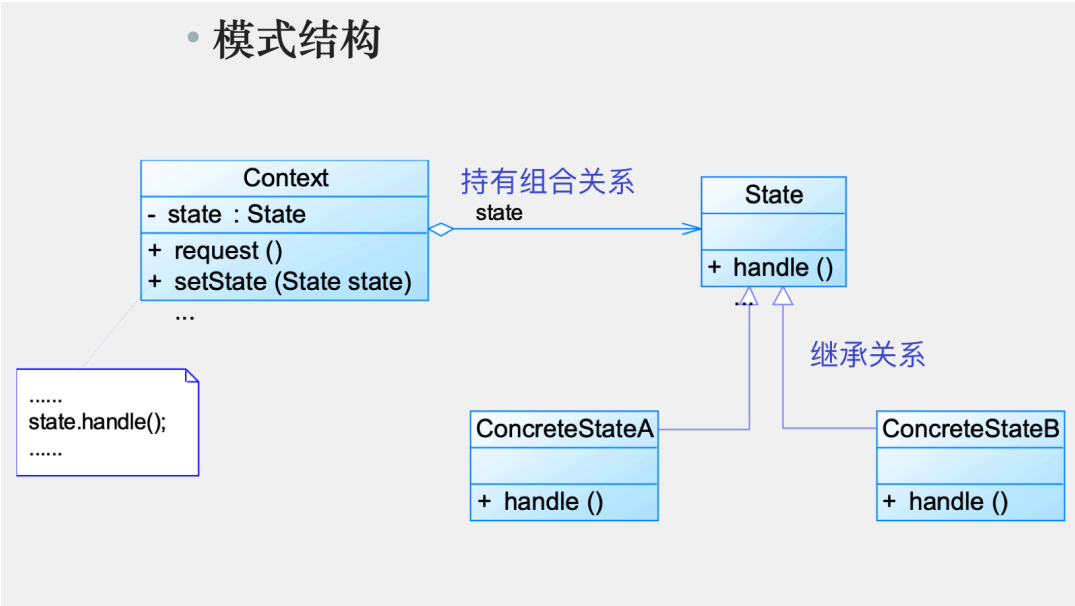


可以用在一些算法上面，比如最小生成树算法持有 Kruskal 和 Prim 两种策略，可以更换不同的算法实现。

状态模式 (State Pattern)

又称：状态对象 (Objects for States)

是对象行为型模式。



Context 类又叫**环境类**，实际上是拥有状态的对象，有时候充当状态管理器 (State Manager) 的角色，可以在环境类中对状态进行切换操作。

优点

- 封装了转换规则
- 枚举可能的状态，可以方便地增加新的状态
- 允许状态转换逻辑与状态对象合成一体
- 可以让多个环境对象共享一个状态对象

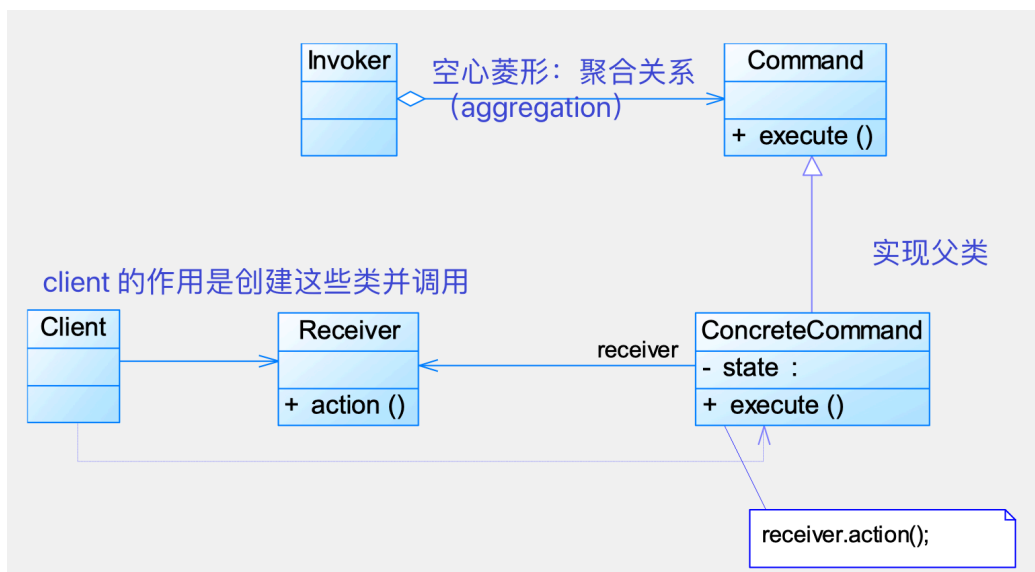
缺点

- 会增加系统类和对象的个数
- 容易造成混乱
- 对“开闭原则”的支持并不太好

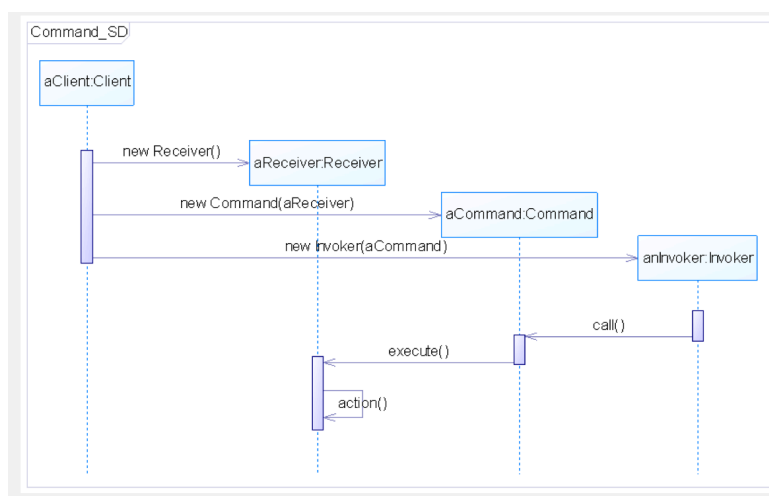
命令模式 (Command Pattern)

又称：

- 动作 (Action) 模式
- 事务 (Transaction) 模式



时序图（顺序图 sequence graph?, 往年题里面似乎也有）



命令模式允许请求的一方和接受的一方独立开来，使得请求的一方不必知道接收的一方的接口，更不必知道请求是怎么被接收，以及操作是否被执行、何时被执行，以及是怎么被执行的。

优点

- 降低系统的耦合度
- 新的命令可以很容易地加入到系统中
- 可以比较容易地设计一个命令队列和宏命令（组合命令）
- 可以方便地实现对请求的 undo 和 redo。

缺点

- 产生过多的具体命令类。

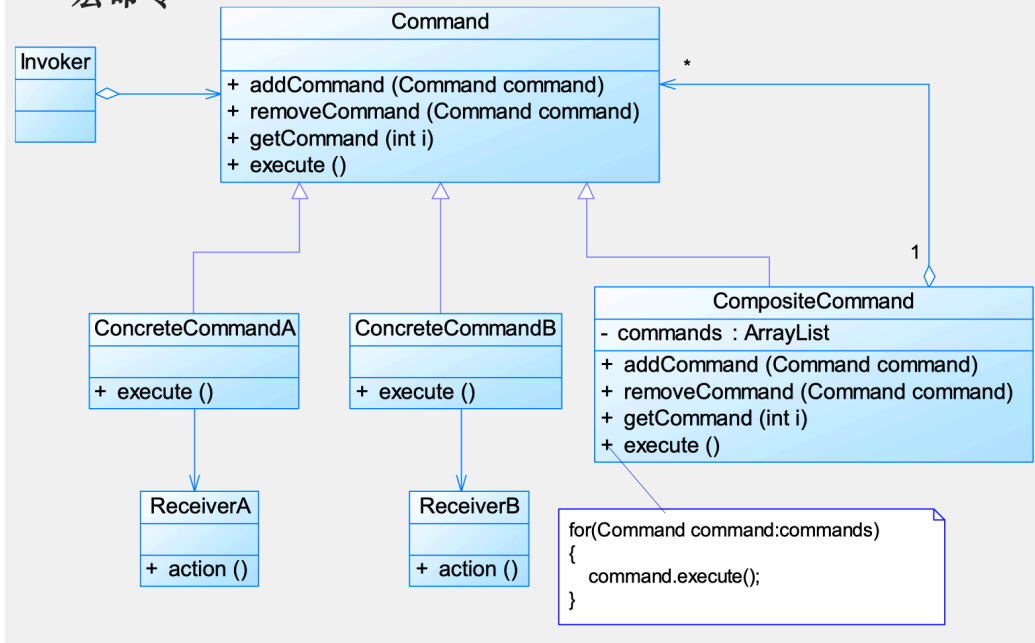
宏命令模式（往年题中有）

又称：组合命令

是命令模式和组合模式联用的产物。

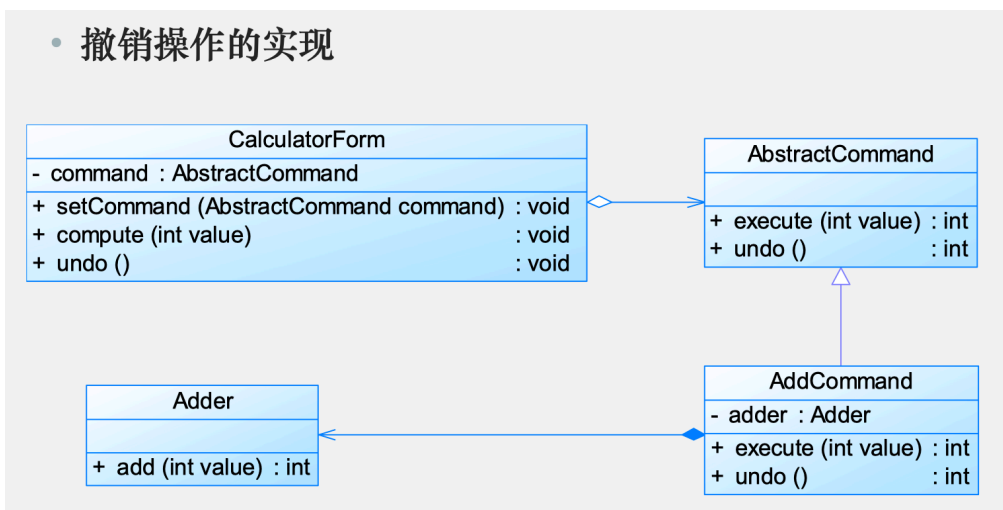
宏命令也是一个具体命令，不过它包含了对其他命令对象的引用，在调用宏命令的 `execute()` 方法时，将递归调用它所包含的每个成员命令的 `execute()` 方法。

• 宏命令



撤销操作

• 撤销操作的实现



感觉是 `new` 了一个 `calculatorForm()` 之后，把这个命令引用先持有着，`undo()` 的时候就重新调用 `adder.add(-value)`，实现 `undo` 的效果，所以这个效果是和具体的操作实现有关的；而且：应该需要一个地方（倾向于在 `CalculatorForm` 中）存放这个操作的 `value` 值。

观察者模式 (Observer Pattern)

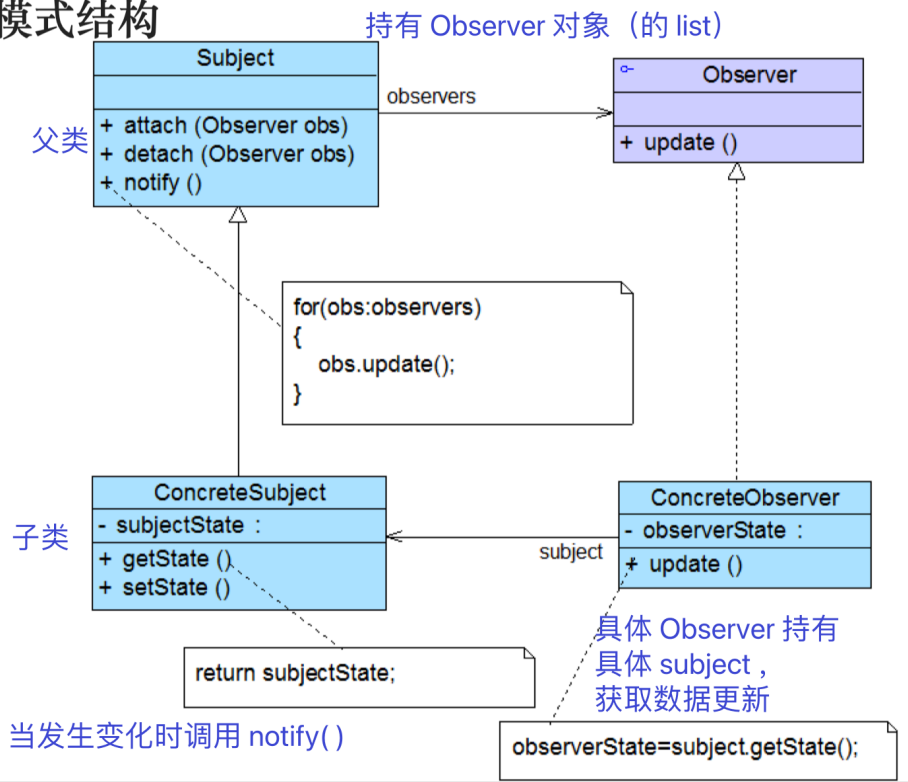
又称：

- 发布-订阅 (Publish/Subscribe) 模式
- 模型-视图 (Model/View) 模式
- 源-监听器 (Source/Listener) 模式
- 从属者 (Dependents) 模式

是一种对象行为型模式。

定义对象间的一种**一对多**的依赖关系，使得每当一个对象状态发生改变时，其相关依赖对象皆得到通知并被自动更新。

• 模式结构



可以通过 `attach()` 和 `detach()` 来更新“订阅”情况，又称“发布-订阅模式”。

松耦合

改变主题或观察者其中一方，并不会影响另一方。只要他们的接口仍被遵守，就可以自由地改变他们。

松耦合是一种很好的状态，很好的目标。

优点

- 可以实现表示层和数据逻辑层的分离
- 在观察目标和观察者之间建立一个抽象的耦合
- 支持广播通信
- 符合“开闭原则”

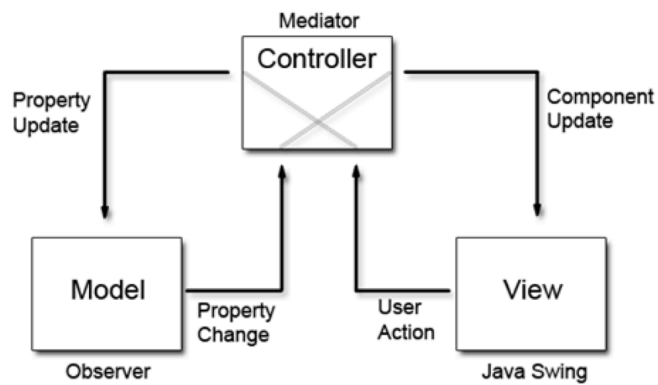
缺点

- 时间开销大
- (可能的) 循环依赖会导致系统崩溃
- 没有相应的机制让观察者知道所观察的目标对象是怎么发生变化的，仅仅只是知道观察目标发生了变化。

MVC 模式

是一种架构模式，而不是设计模式。

MVC模式

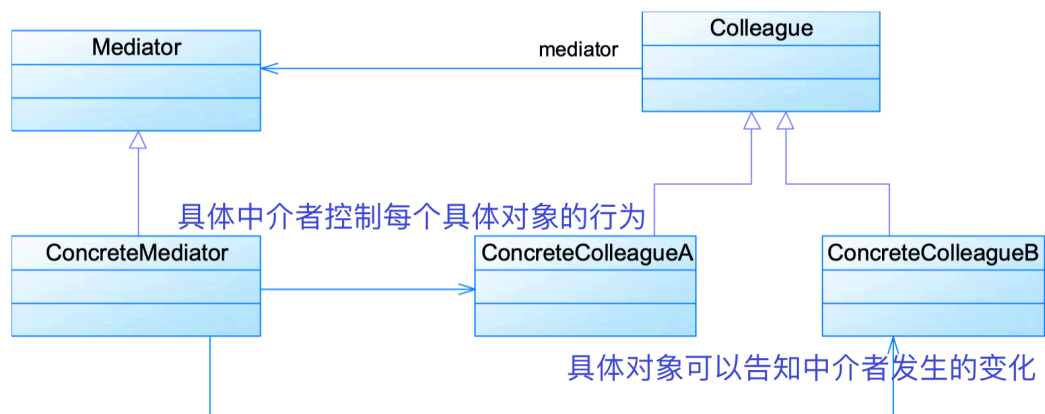


观察者模式可以用来实现 MVC 模式，观察者模式中的观察目标就是 MVC 模式中的模型（Model），而观察者就是 MVC 中的视图（View），控制器（Controller）充当两者之间的中介者（Mediator）。

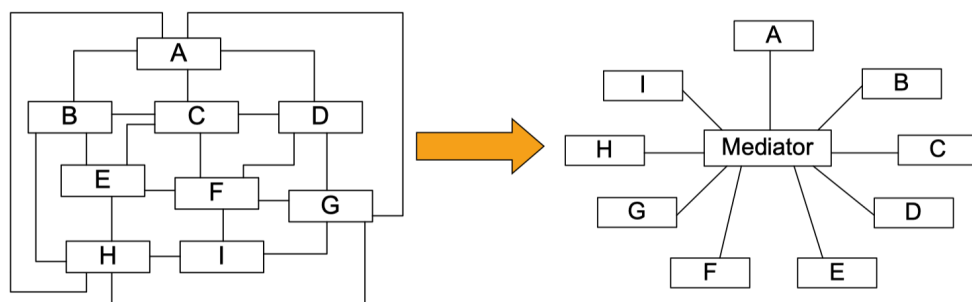
中介者模式 (Mediator Pattern)

又称：调停者模式

是一种对象行为型模式。



- 中介者模式可以使对象之间的关系数量急剧减少：



中介者模式是迪米特法则的一个典型应用。

优点

- 简化了对象之间的交互
- 将各同事解耦
- 减少子类生成
- 可以简化各同事类的设计和实现

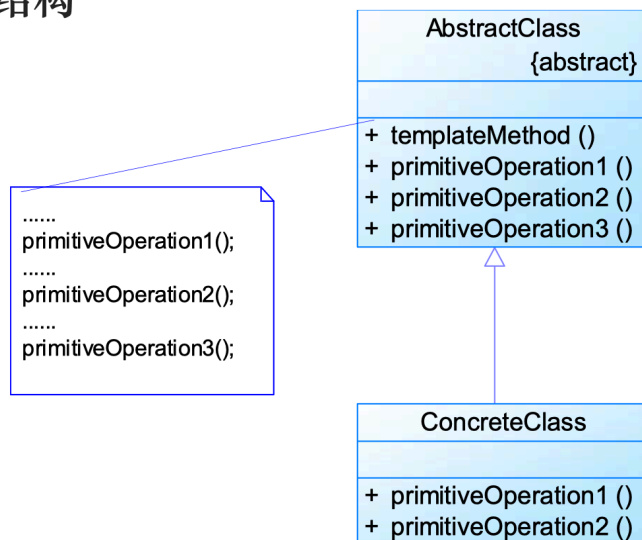
缺点

- 中介者类承担的职责重，导致非常复杂，使得系统难以维护

模板方法模式 (Template Method Pattern)

是一种类行为型模式（也是唯一要考的类行为型模式）。

• 模式结构



感觉跟普通的继承多态没有太大的差别。

在它的结构图中只有类之间的继承关系，没有对象关联关系。

钩子方法 (Hook Method)

• 钩子方法(Hook Method)

```
.....
public void template()
{
    open();
    display();
    if(isPrint())
    {
        print();
    }
}
public boolean isPrint()
{
    return true;
}
.....
```

子类通过重载钩子方法（比如上图中的 `isPrint()`）可以控制父类方法的行为（反向控制）。

关于继承的讨论

模板方法鼓励我们恰当使用继承，是体现继承优势的模式之一。

好莱坞原则

子类不显式调用父类的方法，而是通过覆盖父类的方法来实现这些具体的业务逻辑，父类控制对子类的调用，这种机制被称为**好莱坞原则（Hollywood Principle）**（Don't call us, we'll call you）。

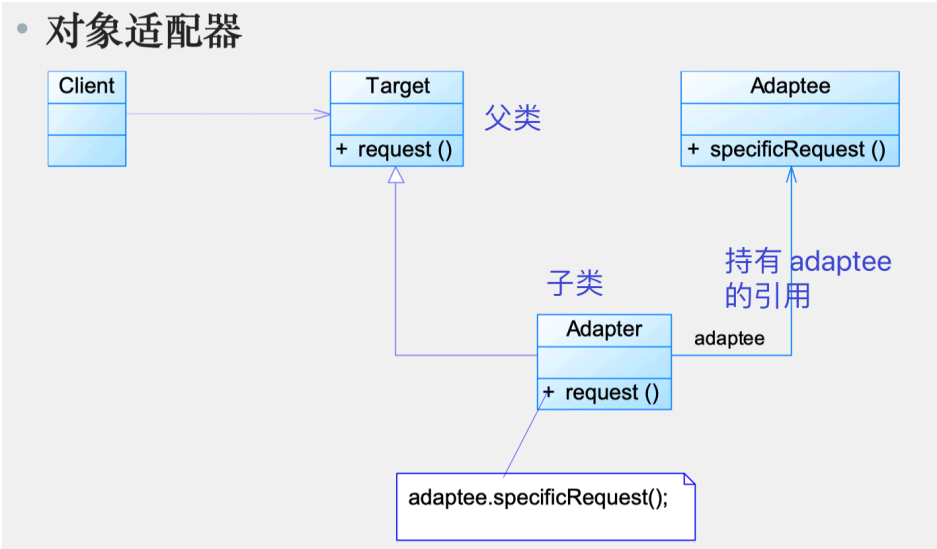
在模板方法模式中，好莱坞原则体现在：子类不需要调用父类，而通过父类来调用子类。

适配器模式（Adapter Pattern）

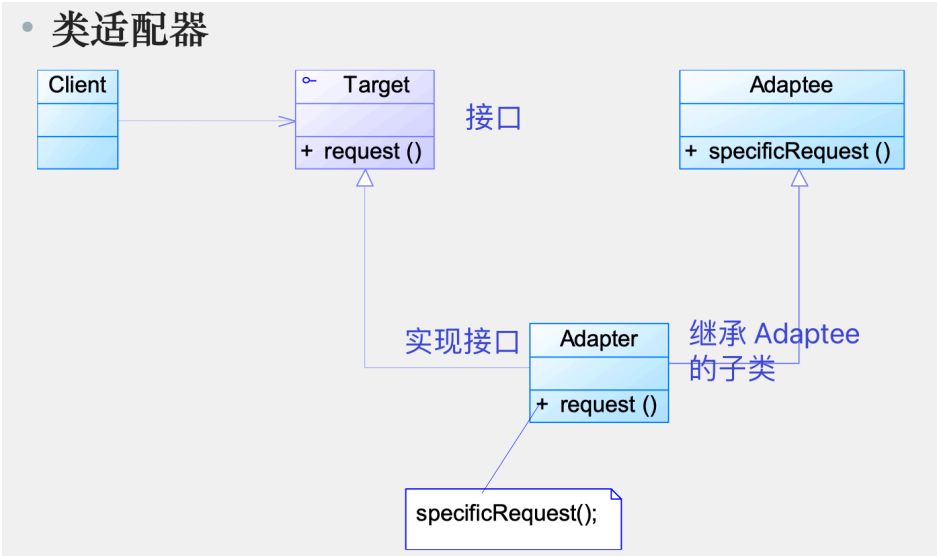
又名：包装器（Wrapper）

既可以作为类结构型模式，也可以作为对象结构性模式。

对象适配器（对象模式）



类适配器（类模式）



或许可以理解成两种不同的实现，但是由于关联 / 继承的不同的关系产生了分类上的不同。

优点

- 适配器模式（包括类和对象适配器模式）的优点
 - 将目标类和适配者类解耦
 - 增加了类的透明性和复用性
 - 灵活性和扩展性都非常好
- 类适配器的优点

- 可以在适配器类中置换一些适配者的方法，使得适配器的灵活性更强。
- 对象适配器的优点
 - 同一个适配器可以把适配器类和它的子类都适配到目标接口

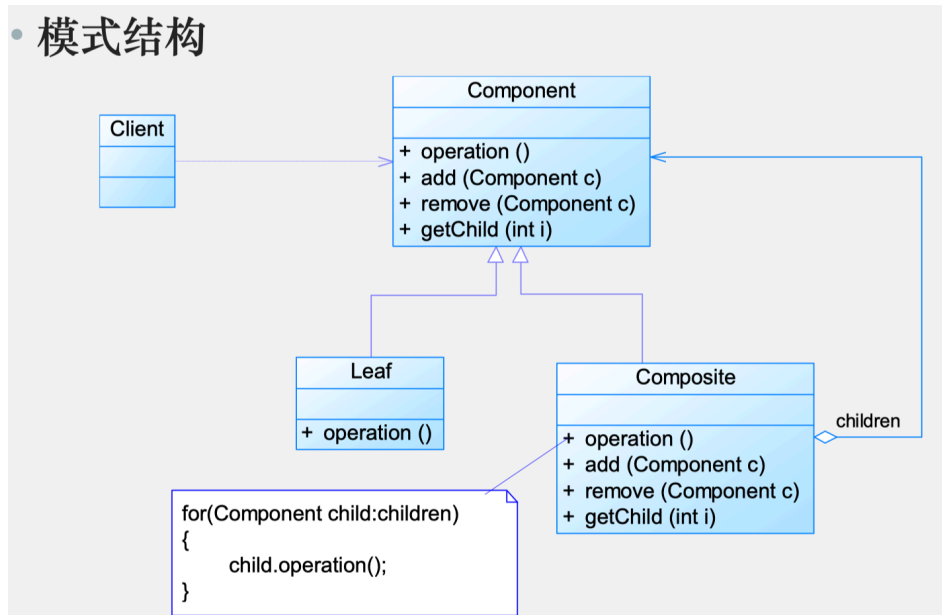
缺点

- 类适配器的缺点
 - Java 和 C# 等不支持多重继承的语言，一次最多只能适配一个适配者类
 - 目标抽象类只能为抽象类，不能为具体类，使用有一定的局限性，不能将一个适配者类和它的子类都适配到目标接口
- 对象适配器的缺点
 - 要想置换适配器类的方法不容易（和类适配器相比）

组合模式（Composite Pattern）

又叫：整体-部分（Part-Whole）模式

是对象行为型模式。



优点

- 可以清楚地定义分层次的复杂对象
- 可以形成复杂的树形结构
- 更容易在组合体内加入对象构件

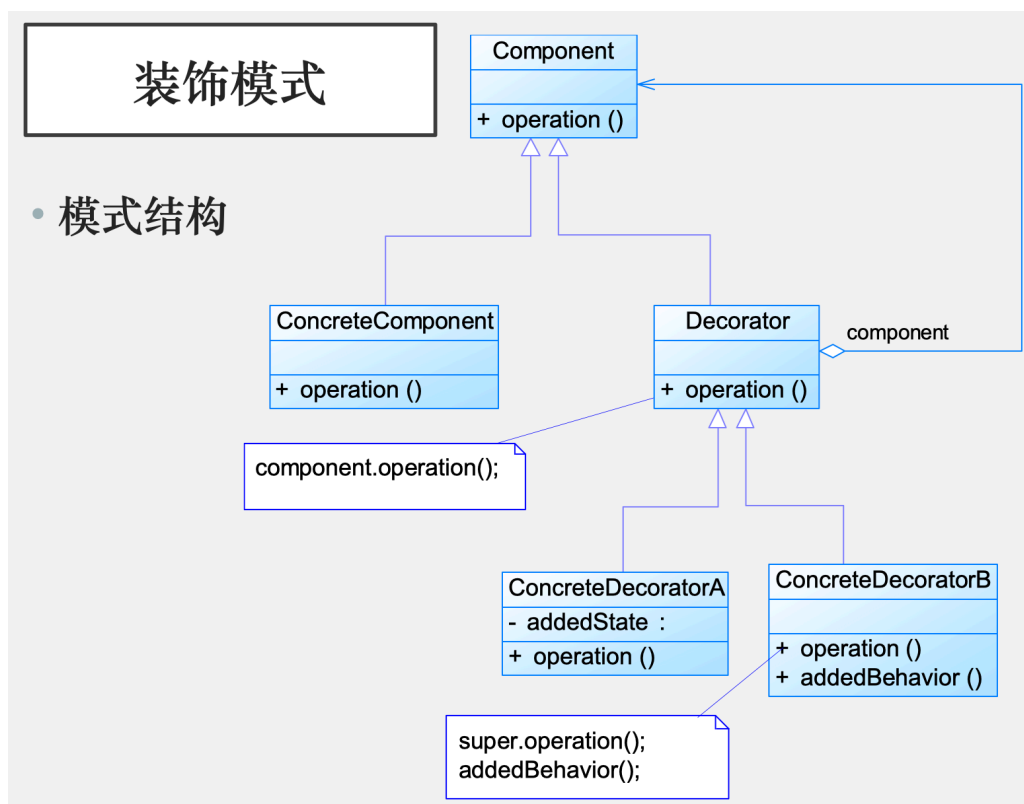
缺点

- 使设计变得更加抽象
- 很难对容器中的构件类型进行限制

装饰模式 (Decorator Pattern)

又称：包装器 (Wrapper)，注意这个别名，这和适配器模式的别名相同

是对象结构型模式。



实际上就像是一个弱化版的组合模式，只是它的递归层次是线性的（可以在一副有画框的画外面多套一个画框），实现方便。

优点

- 装饰模式能比继承提供更多的灵活性
- 通过一种动态的方式来扩展一个对象的功能
- 通过使用不同的具体装饰类以及这些类的排列组合，可以创造出很多不同行为的组合
- 具体构件类与具体装饰类可以独立变化

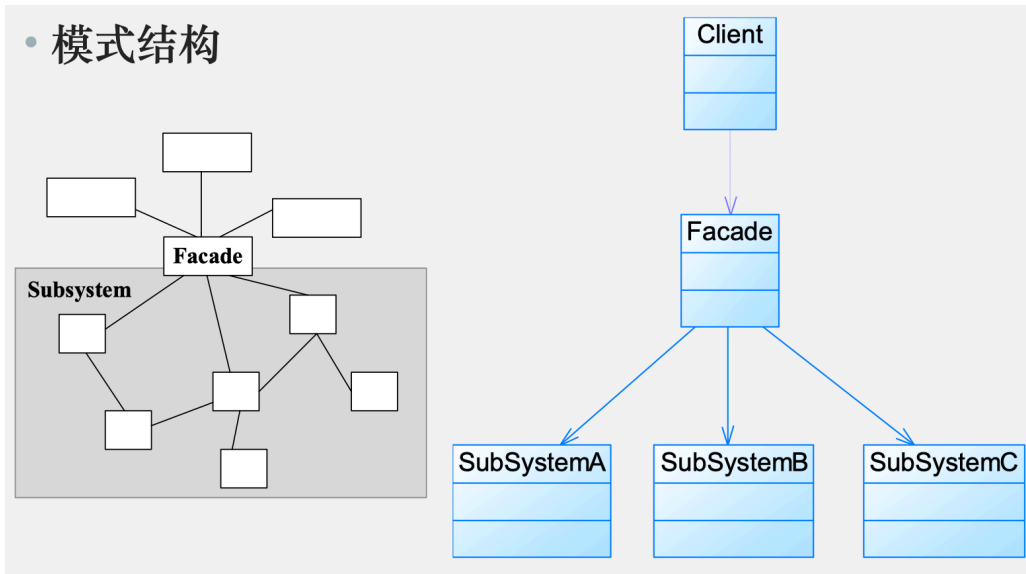
缺点

- 产生很多小对象
- 装饰模式比继承更加易于出错，排错也很困难，对于多次装饰的对象，调试时寻找错误可能需要逐级排查，较为繁琐

外观模式 (Facade Pattern)

是一种对象结构型模式。

• 模式结构



- 根据“**单一职责原则**”，引入外观对象，为子系统的访问提供了一个简单而单一的入口。
- 也是“**迪米特法则**”的体现，通过引入一个新的外观类可以降低原有系统的复杂度，同时降低客户类与子系统类的耦合度。

优点

- 减少了客户处理的对象数目并且使得子系统使用起来更加容易
- 实现了子系统和客户之间的松耦合关系
- 降低了大型软件系统中的编译依赖性，并简化了系统在不同平台之间的移植过程
- 只提供了一个访问子系统的统一入口，并不影响用户直接使用子系统类（恶意用户绕过使用外观类的约定直接访问子系统）

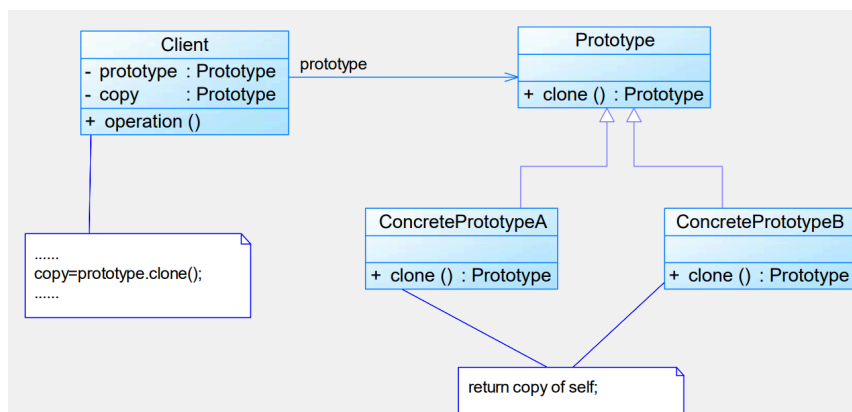
缺点

- 不能很好地限制客户使用子系统类
- 不引入抽象外观类的情况下，增加新的子系统可能需要修改外观类或客户端的源代码，**违反了“开闭原则”**

不要试图通过外观类为子系统增加新行为

原型模式 (Prototype Pattern)

是一种对象创建型模式。



- 用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象。原型模式允许一个对象再创建另外一个可定制的对象，无须知道任何创建的细节。
- 基本工作原理是通过将一个原型对象传给那个要发动创建的对象，这个要发动创建的对象通过请求原型对象拷贝原型自己来实现创建过程。

优点

- 当创建新的对象实例较为复杂时，使用原型模式可以简化对象的创建过程，通过一个已有实例可以提高新实例的创建效率
- 可以动态增加或减少产品类
- 原型模式提供了简化的创建结构
- 可以使用深克隆的方式保存对象的状态

缺点

- 需要为每一个类配备一个克隆方法，而且这个克隆方法需要对类的功能进行通盘考虑，这对全新的类来说不是很难，但对已有的类进行改造时，不一定是件容易的事，必须修改其源代码，**违背了“开闭原则”**
- 在实现深克隆时需要编写较为复杂的代码

表驱动

表驱动法是一种编程模式（scheme），使用从表里面查找信息而不使用逻辑信息（if 和 else）

优点

- 表驱动法适用于复杂的逻辑；
- 可以将复杂逻辑从代码中独立出来，以便于单独维护。

从表中访问条目的方法

直接访问（Direct Access）

通过索引值（下标）访问表中元素。

索引访问（Indexed Access）

一种间接访问的技术，先从索引表中查询到主表的下标，然后进去访问。

- 优点
 - 若主表中数据很大，索引表可以节省很多空间（例如，主表十分稀疏）；
 - 操作索引中的数据比操作主表中的数据更加便捷；
 - 编写到表里面的数据比编写到代码中的数据更加容易维护（表驱动方法的优点，不是索引访问的优点）

阶梯访问（Stair-step Access）

表中存储端点。

- 把每一区间的上限写入一张表里，然后写一个循环，按照各区间的上限来检查分数
 - 当分数第一次超过某个区间的上限时，你就知道相应的等级了
 - 区间表：{ 50.0, 65.0, 75.0, 90.0, 100.0 }
- 是一种对象创建型模式